W1

Class Diagrams & Design Principles (SOLID) - 2

Data Structures & Object-Oriented Programming



Table of Contents

- LSP (Liskov Substitution Principle)
- การทำงานร่วมกันของ SOLID

LSP (Liskov Substitution Principle)

LSP (Liskov Substitution Principle)

- “Subclasses must be substitutable for their parent classes.”
  - หลักการนี้ตั้งชื่อตาม Barbara Liskov ผู้เสนอแนวคิดนี้ในปี 1987 และเป็นหลักการที่เชื่อมโยงโดยตรงกับ Inheritance
  - ความหมายของ “substitutable” (แทนที่กันได้) คือ ถ้า function ทำงานถูกต้องกับ object ของ parent class มันต้องทำงานถูกต้องกับ object ของ subclass ทุกตัวด้วย โดยไม่ต้องแก้ function นั้นเลยแม้แต่บรรทัดเดียว
- เมื่อไรก็ตามที่ code ส่วนหนึ่งเขียนขึ้นมาโดยคาดหวังว่าจะได้รับ BankAccount เข้ามาทำงานด้วย
  - ไม่ว่า object ที่ส่งเข้ามาจริงๆ จะเป็น BankAccount ตรงๆ หรือเป็น SavingsAccount ที่ inherit มา code ส่วนนั้นก็ต้องทำงานถูกต้องเหมือนเดิมทุกประการ
  - ถ้า code พังหรือให้ผลลัพธ์ผิดเมื่อได้รับ subclass เข้ามาแทน parent นั่นคือสัญญาณของการละเมิด LSP

ตัวอย่างคลาสสิก — Square/Rectangle

```python
class Rectangle:
    def __init__(self, width: float, height: float):
        self._width = width
        self._height = height

    @property
    def width(self) -> float:
        return self._width

    @width.setter
    def width(self, value: float) -> None:
        self._width = value

    @property
    def height(self) -> float:
        return self._height

    @height.setter
    def height(self, value: float) -> None:
        self._height = value

    def area(self) -> float:
        return self._width * self._height
```

```python
class Square(Rectangle):
    """ดูสมเหตุสมผล — Square IS-A Rectangle? แต่ละเมิด LSP"""

    @Rectangle.width.setter
    def width(self, value: float) -> None:
        self._width = value
        self._height = value    # Square ต้องให้ด้านเท่ากันเสมอ

    @Rectangle.height.setter
    def height(self, value: float) -> None:
        self._width = value
        self._height = value
```

- ในทาง geometry : Square เป็น Rectangle ชนิดหนึ่งจริงๆ
  - การ inherit ดูสมเหตุสมผลอย่างยิ่ง
- ปัญหาอยู่ที่ setter ของ Square ถูกเขียนให้ แก้ไข attribute ทั้งสองพร้อมกัน
  - parent (Rectangle) ไม่เคยมีพฤติกรรมแบบนี้
  - ไม่มีใครที่ใช้ Rectangle คาดคิดว่าการตั้งค่า width เพียงอย่างเดียวจะไปเปลี่ยน height ด้วย

กลไกที่ทำให้เกิดผลจริง

- “สัญญา” (contract) ระหว่าง parent กับ child
  - เมื่อ parent class กำหนดพฤติกรรมบางอย่างไว้ (เช่น Rectangle ที่ให้ width และ height เปลี่ยนแปลงได้อิสระจากกัน) นั่นถือเป็นสัญญาที่ผู้ใช้งาน Rectangle ทุกคนคาดหวังไว้ล่วงหน้า
- เมื่อ child class inherit มาแล้ว override method ในลักษณะที่ทำลายสัญญาเดิม (เช่นบังคับให้ width กับ height ต้องเท่ากันเสมอ)
  - แม้ว่าในเชิงแนวคิดจะดู “สมเหตุสมผล” แค่ไหนก็ตาม (สี่เหลี่ยมจัตุรัสก็เป็นสี่เหลี่ยมผืนผ้าชนิดหนึ่งจริงๆ)
  - แต่ในทาง code มันคือการทำลายสัญญาที่ผู้ใช้งาน parent class วางใจไว้ และ code ใดๆ ที่เขียนขึ้นโดยอาศัยสัญญานั้นจะพังทันทีเมื่อได้รับ child class เข้ามาแทน

จำลองสถานการณ์: ทดสอบ LSP Violation

```python
def assert_rectangle_invariant(rect: Rectangle) -> None:
    """Test ที่ถูกต้องสำหรับ Rectangle ทุกตัว"""
    rect.width = 4
    rect.height = 5
    assert rect.area() == 20, f"Expected 20, got {rect.area()}"

r = Rectangle(3, 3)
assert_rectangle_invariant(r)   # ✓ ผ่าน — area = 4 × 5 = 20

s = Square(3)
assert_rectangle_invariant(s)   # ✗ ล้มเหลว! — area = 5 × 5 = 25 ≠ 20
# Square แก้ width ทำให้ height เปลี่ยนตาม ทำลาย invariant ของ Rectangle
```

- ฟังก์ชัน assert_rectangle_invariant() ถูกเขียนขึ้นโดยอาศัย สัญญา ของ Rectangle ล้วนๆ คือ “ตั้งค่า width แล้ว height ไม่เปลี่ยนตาม” มันไม่ได้รู้จักว่าจะมี Square อยู่ในระบบเลย แต่เมื่อได้รับ Square เข้ามาแทนที่ Rectangle (ซึ่งควรทำได้เพราะ Square inherit จาก Rectangle) ผลลัพธ์กลับผิดพลาดทันที
- หัวใจของปัญหา LSP — ไม่ใช่ Square ผิดในตัวมันเอง แต่ Square ทำให้ code ซึ่งเคยทำงานถูกต้องกับ Rectangle กลับทำงานผิดเมื่อสลับมาใช้ Square แทน ทั้งที่ไม่มีใครไปแก้ไข assert_rectangle_invariant() เลยแม้แต่บรรทัดเดียว

แก้ไขโดย Refactor Hierarchy

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

class Rectangle(Shape):    # Rectangle เป็น concrete class อิสระ
    def __init__(self, width: float, height: float):
        self._width = width
        self._height = height
    # width และ height เปลี่ยนได้อิสระจากกัน

    def area(self) -> float:
        return self._width * self._height

class Square(Shape):       # Square เป็น concrete class อิสระ ไม่ใช่ Rectangle
    def __init__(self, side: float):
        self._side = side

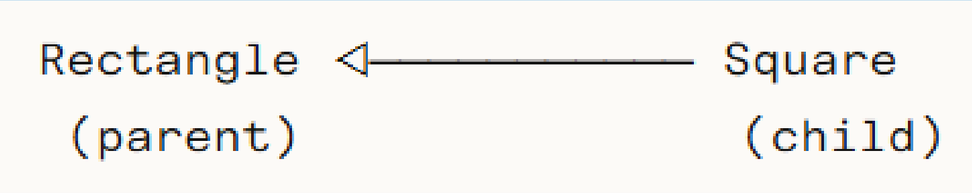
    def area(self) -> float:
        return self._side ** 2
```

- ทางออกที่ถูกต้อง คือการ ยอมรับว่า Square ไม่ควร inherit จาก Rectangle ตั้งแต่แรก
  - ทั้งคู่เป็น concrete class ที่เป็นอิสระจากกัน
  - มี Shape เป็น Abstract Class ร่วมกันในระดับเดียวกัน

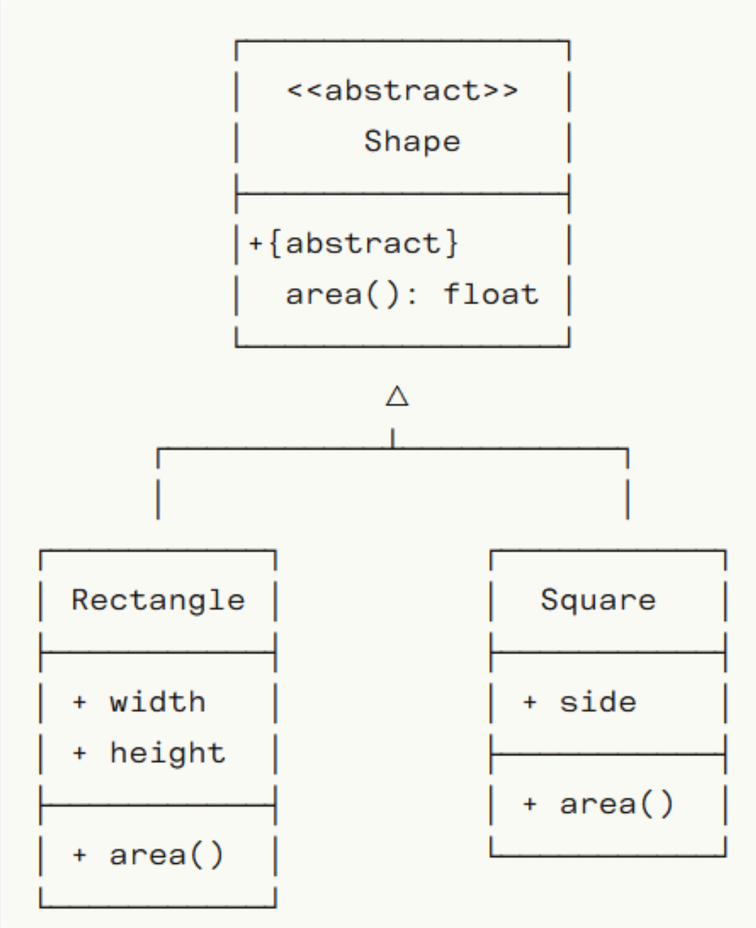
- เมื่อทั้งคู่เป็น subclass ของ Shape ซึ่งอยู่ใน level เดียวกัน
  - ไม่มีใครทำลาย invariant ของอีกฝ่ายอีกต่อไป

UML ก่อนและหลัง Refactor

ก่อน Refactor — โครงสร้างที่ดูสมเหตุสมผลแต่ละเมิด LSP



หลัง Refactor — โครงสร้างที่ถูกต้องตาม LSP



- diagram แรกบอกว่า “Square เป็นชนิดพิเศษของ Rectangle”
  - คนเขียน code คาดหวัง behavior แบบ Rectangle จาก Square ด้วย
- diagram สองบอกอย่างตรงไปตรงมาว่า “ทั้งคู่เป็นรูปทรงคนละแบบที่บังเอิญคำนวณพื้นที่ได้เหมือนกัน”
  - ไม่เกิดความคาดหวังผิดๆ

ประโยชน์ที่ได้จริงในการทำงาน

- Polymorphism ทำงานได้อย่างน่าเชื่อถือ 100% เมื่อ hierarchy ออกแบบถูกต้องตาม LSP
  - โค้ดที่เขียนไว้สำหรับ parent จะทำงานถูกต้องกับทุก subclass เสมอ แม้จะมีเพิ่มเติมมาในอนาคต
  - ไม่ต้องกังวลว่าจะมี subclass ตัวไหนแอบทำลาย behavior ที่คาดหวังไว้
- ลดความจำเป็นในการเช็ค type ก่อนเรียกใช้งาน เพราะมั่นใจว่าทุก subclass แทนที่ parent ได้จริง
- การทดสอบมีความน่าเชื่อถือสูงขึ้น
  - เมื่อเขียน unit test ให้กับ parent class แล้ว มั่นใจได้ว่า subclass ทุกตัวที่ผ่าน test เดียวกันจะทำงานถูกต้องในทุกที่ที่ parent เคยถูกใช้งาน
  - ไม่ต้องเขียน test แยกสำหรับทุก edge case ของทุก subclass ใหม่ทั้งหมด

กับดักที่ต้องระวัง

- ในทางปฏิบัติจริง การละเมิด LSP มักเกิดในรูปแบบที่สังเกตยากกว่าตัวอย่างที่นำเสนอ
  - เช่น การ override method แล้วเพิ่มเงื่อนไขที่ parent ไม่เคยมี โดยเฉพาะการ throw exception ในสถานการณ์ที่ parent ไม่เคย throw

```python
class Bird:
    def fly(self) -> None:
        print("กำลังบิน")

class Penguin(Bird):
    def fly(self) -> None:
        raise NotImplementedError("เพนกวินบินไม่ได้!")  # ← LSP violation
```

- ในเชิงชีววิทยา เพนกวินก็เป็นนกจริงๆ การเขียน class Penguin(Bird): จึงดูสมเหตุสมผล
- แต่ปัญหาคือ Bird.fly() สร้างสัญญาไว้ว่า “เรียกแล้วนกจะบิน” โดยไม่มีเงื่อนไขใดๆ
- ส่วน Penguin.fly() กลับ เพิ่มเงื่อนไขใหม่ขึ้นมาเอง คือ “เรียกแล้ว error เสมอ” ซึ่งเป็นการทำลายสัญญาเดิมอย่างสมบูรณ์

กับดักที่ต้องระวัง (2)

- ลองพิจารณาผลกระทบ

```python
def make_all_birds_fly(birds: list[Bird]) -> None:
    for bird in birds:
        bird.fly()    # ฟังก์ชันนี้เขียนขึ้นโดยเชื่อสัญญาของ Bird

birds = [Bird(), Bird(), Penguin()]
make_all_birds_fly(birds)
# Bird ตัวแรกและตัวที่สองบินได้ปกติ
# แต่พอถึง Penguin โปรแกรมทั้งหมด crash ทันที
```

- ฟังก์ชัน make_all_birds_fly() ไม่เคยรู้จัก Penguin เลยตอนที่เขียนขึ้น
- มันแค่เชื่อในสัญญาของ Bird แต่กลับพังเพราะ subclass ตัวหนึ่งแอบเปลี่ยนกติกา

วิธีแก้ที่ถูกต้องคือออกแบบ hierarchy ใหม่

- **แยกสัตว์ที่บินได้กับบินไม่ได้ออกจากกันตั้งแต่ระดับโครงสร้าง**
  - ให้ FlyingBird และ NonFlyingBird เป็น subclass ของ Bird
  - โดยมีเพียง FlyingBird เท่านั้นที่มี fly() method อยู่ใน class ของตัวเอง
  - ส่วน Bird (base) ไม่ควรมี fly() เป็นสัญญาตั้งแต่ต้น เพื่อไม่ให้ code ใดๆ คาดหวังว่านกทุกตัวต้องบินได้

```python
class Bird:
    """ไม่มี fly() ในระดับนี้ — ไม่ใช่นกทุกตัวบินได้"""
    def eat(self) -> None:
        print("กำลังกิน")

class FlyingBird(Bird):
    def fly(self) -> None:
        print("กำลังบิน")
```

```python
class NonFlyingBird(Bird):
    def walk(self) -> None:
        print("กำลังเดิน")

class Sparrow(FlyingBird):
    pass

class Penguin(NonFlyingBird):
    pass
```

โครงสร้างใหม่นี้ทำให้ไม่มีทางเรียก penguin.fly() ได้ตั้งแต่ต้น เพราะ Penguin ไม่เคยมี method นั้นอยู่ในสายการสืบทอดของมัน

เมื่อ SOLID ทำงานร่วมกัน: NotificationService

เมื่อ SOLID ทำงานร่วมกัน: NotificationService

- ในการทำงานจริง code หนึ่งชิ้นมักละเมิดหลายหลักการพร้อมกัน และการ refactor หนึ่งครั้งมักแก้ปัญหาหลายข้อไปพร้อมกัน
- ตัวอย่างต่อไปนี้ : code ที่ละเมิดทั้ง SRP และ OCP พร้อมกันในจุดเดียว และการ refactor ครั้งเดียวที่แก้ทั้งสองปัญหาได้พร้อมกัน โดยอาศัยกลไก Abstraction

```python
class NotificationService:
    """God class ที่รู้เรื่องทุกอย่างมากเกินไป"""

    def notify(self, recipient: str, message: str, channel: str) -> None:
        # SRP violation: notification logic ปนกับ format logic
        if channel == "email":
            formatted = f"Subject: Notification\n\n{message}"
            print(f"[Email] To: {recipient}\n{formatted}")
        elif channel == "sms":
            # SMS จำกัด 160 ตัวอักษร — ต้องตัด
            truncated = message[:160]
            print(f"[SMS] To: {recipient}: {truncated}")
        elif channel == "line":
            print(f"[Line] @{recipient}: {message}")
        # OCP violation: เพิ่ม channel ใหม่ → ต้องแก้ method นี้เสมอ

        # SRP violation: logging ปนอยู่กับ notification
        with open("notifications.log", "a") as f:
            f.write(f"{channel}: {recipient}: {message}\n")
```

- จุดที่ 1 — ละเมิด SRP เพราะ method เดียวมีเหตุผลในการเปลี่ยนแปลงอยู่ 2 อย่าง
  - วิธี format ข้อความสำหรับแต่ละช่องทาง
  - วิธีบันทึกประวัติการแจ้งเตือน” (ส่วน with open(…))
  - ถ้าฝ่ายหนึ่งต้องการเปลี่ยนรูปแบบ log จาก text file เป็น database และอีกฝ่ายต้องการเพิ่มช่องทางแจ้งเตือนใหม่ ทั้งสองคนต้องมาแก้ไข method เดียวกันนี้

เมื่อ SOLID ทำงานร่วมกัน: NotificationService

- ในการทำงานจริง code หนึ่งชิ้นมักละเมิดหลายหลักการพร้อมกัน และการ refactor หนึ่งครั้งมักแก้ปัญหาหลายข้อไปพร้อมกัน
- ตัวอย่างต่อไปนี้ : code ที่ละเมิดทั้ง SRP และ OCP พร้อมกันในจุดเดียว และการ refactor ครั้งเดียวที่แก้ทั้งสองปัญหาได้พร้อมกัน โดยอาศัยกลไก Abstraction

```python
class NotificationService:
    """God class ที่รู้เรื่องทุกอย่างมากเกินไป"""

    def notify(self, recipient: str, message: str, channel: str) -> None:
        # SRP violation: notification logic ปนกับ format logic
        if channel == "email":
            formatted = f"Subject: Notification\n\n{message}"
            print(f"[Email] To: {recipient}\n{formatted}")
        elif channel == "sms":
            # SMS จำกัด 160 ตัวอักษร — ต้องตัด
            truncated = message[:160]
            print(f"[SMS] To: {recipient}: {truncated}")
        elif channel == "line":
            print(f"[Line] @{recipient}: {message}")
        # OCP violation: เพิ่ม channel ใหม่ → ต้องแก้ method นี้เสมอ

        # SRP violation: logging ปนอยู่กับ notification
        with open("notifications.log", "a") as f:
            f.write(f"{channel}: {recipient}: {message}\n")
```

- จุดที่ 2 — ละเมิด OCP
  - ทุกครั้งที่ต้องเพิ่มช่องทางใหม่ ต้องกลับมา “เปิด” แก้ไข method notify() นี้เสมอ
- สังเกตว่าทั้งสองปัญหานี้ เกิดจากจุดเดียวกัน คือการที่ logic หลายอย่างถูกยัดรวมไว้ในฟังก์ชันเดียว
- การ refactor เพื่อแก้ SRP มักจะแก้ OCP ไปพร้อมกันด้วย โดยธรรมชาติ

จำลองสถานการณ์: เพิ่มช่องทางที่สี่

- ต้องเพิ่มการแจ้งเตือนผ่าน Push Notification เข้าไปในระบบเดิม
- สิ่งที่ต้องทำคือเปิด NotificationService.notify() แล้วเพิ่ม elif channel == "push": เข้าไป
  - การแก้ไขนี้เกิดขึ้นในไฟล์เดียวกับที่จัดการทั้ง email, sms, line และ logging ทั้งหมด ทำให้ผู้พัฒนาต้องอ่านและเข้าใจ logic ทั้งหมดก่อนจะมั่นใจได้ว่าการเพิ่มเงื่อนไขใหม่นี้จะไม่กระทบช่องทางเดิมที่ทำงานอยู่แล้ว

หลัง Refactor — SRP และ OCP ถูกแก้พร้อมกัน

```python
class NotificationChannel(ABC):
    """Contract สำหรับทุก notification channel"""
    @abstractmethod
    def send(self, recipient: str, message: str) -> None:

    @abstractmethod
    def describe(self) -> str: ...


class EmailChannel(NotificationChannel):
    def send(self, recipient: str, message: str) -> None:
        formatted = f"Subject: Notification\n\n{message}"
        print(f"[Email] To: {recipient}\n{formatted}")

    def describe(self) -> str:
        return "Email"
```

```python
class SMSChannel(NotificationChannel):
    MAX_LENGTH = 160

    def send(self, recipient: str, message: str) -> None:
        truncated = message[:self.MAX_LENGTH]
        print(f"[SMS] To: {recipient}: {truncated}")

    def describe(self) -> str:
        return "SMS"


class LineChannel(NotificationChannel):
    def send(self, recipient: str, message: str) -> None:
        print(f"[Line] @{recipient}: {message}")

    def describe(self) -> str:
        return "Line"
```

- NotificationChannel แก้ปัญหา OCP เพราะเป็นสัญญากลางที่ทำให้เพิ่มช่องทางใหม่ได้โดยไม่แก้ code เดิม
- EmailChannel, SMSChannel, LineChannel แก้ปัญหา SRP เพราะแต่ละ class รับผิดชอบการ format และส่งข้อความสำหรับช่องทางของตัวเองเพียงอย่างเดียว

หลัง Refactor — SRP และ OCP ถูกแก้พร้อมกัน

```python
# SRP: Logger มี responsibility เดียว — บันทึก log
class NotificationLogger:
    def __init__(self, log_file: str = "notifications.log"):
        self._log_file = log_file

    def log(self, channel_name: str, recipient: str, message: str) -> None:
        with open(self._log_file, "a") as f:
            f.write(f"{channel_name}: {recipient}: {message}\n")


# SRP: Service ทำหน้าที่ orchestrate เท่านั้น ไม่รู้รายละเอียดของ channel ใดๆ
class NotificationService:
    def __init__(self, logger: NotificationLogger):
        self._logger = logger

    def notify(self, recipient: str, message: str,
               channel: NotificationChannel) -> None:
        channel.send(recipient, message)
        self._logger.log(channel.describe(), recipient, message)
```

- NotificationLogger แก้ปัญหา SRP เพราะแยก responsibility เรื่อง “การบันทึก log” ออกจาก “การส่งข้อความ”
- NotificationService แก้ปัญหา SRP โดยเหลือหน้าที่เพียงอย่างเดียวคือ orchestrate (ประสานงาน) ระหว่างการส่งข้อความกับการบันทึก log
  - มันไม่รู้เลยว่า channel ที่รับเข้ามาเป็นช่องทางใด รู้เพียงว่ามันมี send() และ describe() ตามสัญญาที่ NotificationChannel กำหนดไว้
- แก้ SRP ด้วยเพราะ NotificationService.notify() ทำงานถูกต้องกับทุก subclass ของ NotificationChannel ได้อย่างสมบูรณ์โดยไม่ต้องเช็ค type เลย

ทดสอบและผลลัพธ์

```
logger = NotificationLogger()
service = NotificationService(logger)

channels = [EmailChannel(), SMSChannel(), LineChannel()]
for ch in channels:
    service.notify("alice", "Your order is ready.", ch)
```

Expected Output:

```
[Email] To: alice
Subject: Notification

Your order is ready.
[SMS] To: alice: Your order is ready.
[Line] @alice: Your order is ready.
```

- loop นี้เรียก service.notify() ด้วย object สามชนิดที่ต่างกัน (EmailChannel, SMSChannel, LineChannel)
- แต่ notify() เขียนโค้ดเพียงครั้งเดียวและทำงานถูกต้องกับทั้งสามชนิด — นี่คือ Polymorphism ที่ยืนอยู่บนรากฐานของ LSP

ทดสอบ OCP: เพิ่มช่องทางที่สี่โดยไม่แก้ของเดิม

สถานการณ์ที่ต้องเพิ่ม Push Notification ด้วยโครงสร้างใหม่

```
# เพิ่ม PushChannel → ไม่ต้องแก้ NotificationService, NotificationLogger,
# หรือ EmailChannel/SMSChannel/LineChannel แม้แต่บรรทัดเดียว
class PushChannel(NotificationChannel):
    def send(self, recipient: str, message: str) -> None:
        print(f"[Push] Device {recipient}: {message}")

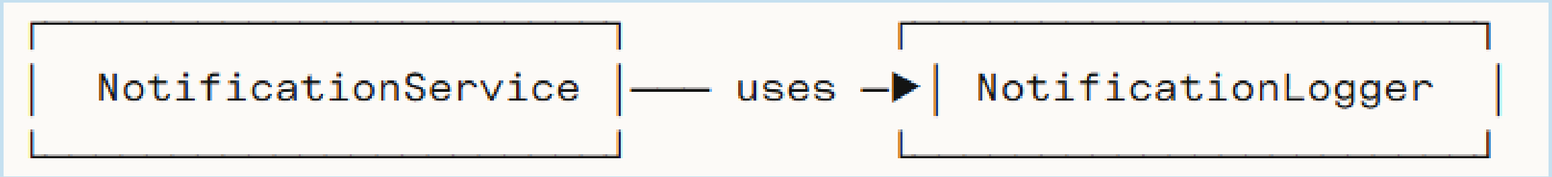
    def describe(self) -> str:
        return "Push Notification"

# ใช้งานได้ทันทีโดยไม่ต้องแก้ loop เดิมเลย
channels.append(PushChannel())
for ch in channels:
    service.notify("alice", "Your order is ready.", ch)
```

การเพิ่มช่องทางใหม่ครั้งนี้ เป็นการเพิ่มไฟล์ใหม่ล้วนๆ ไม่มีการแก้ไข NotificationService, NotificationLogger, หรือ channel เดิมใดๆ เลยแม้แต่บรรทัดเดียว

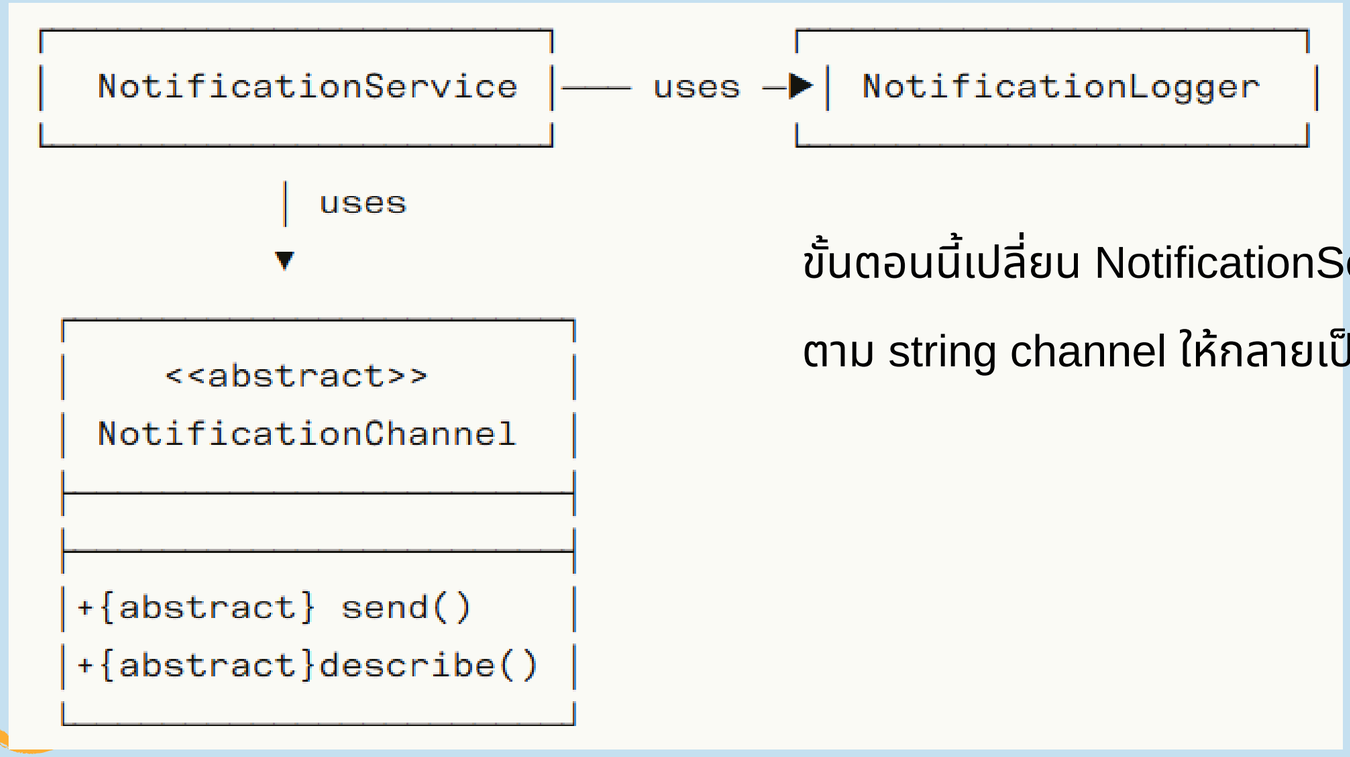
UML ของ Design — สร้างทีละขั้น

ขั้นที่ 1 — เริ่มจากการแยก Responsibility ด้าน Logging ออกมาก่อน (แก้ SRP จุดที่ 1)



ทำให้ “เหตุผลในการเปลี่ยนเรื่อง log” แยกออกจาก “เหตุผลในการเปลี่ยนเรื่องช่องทางส่งข้อความ” ได้แล้ว แต่ปัญหาเรื่อง if-elif chain (OCP) ยังคงอยู่ใน NotificationService

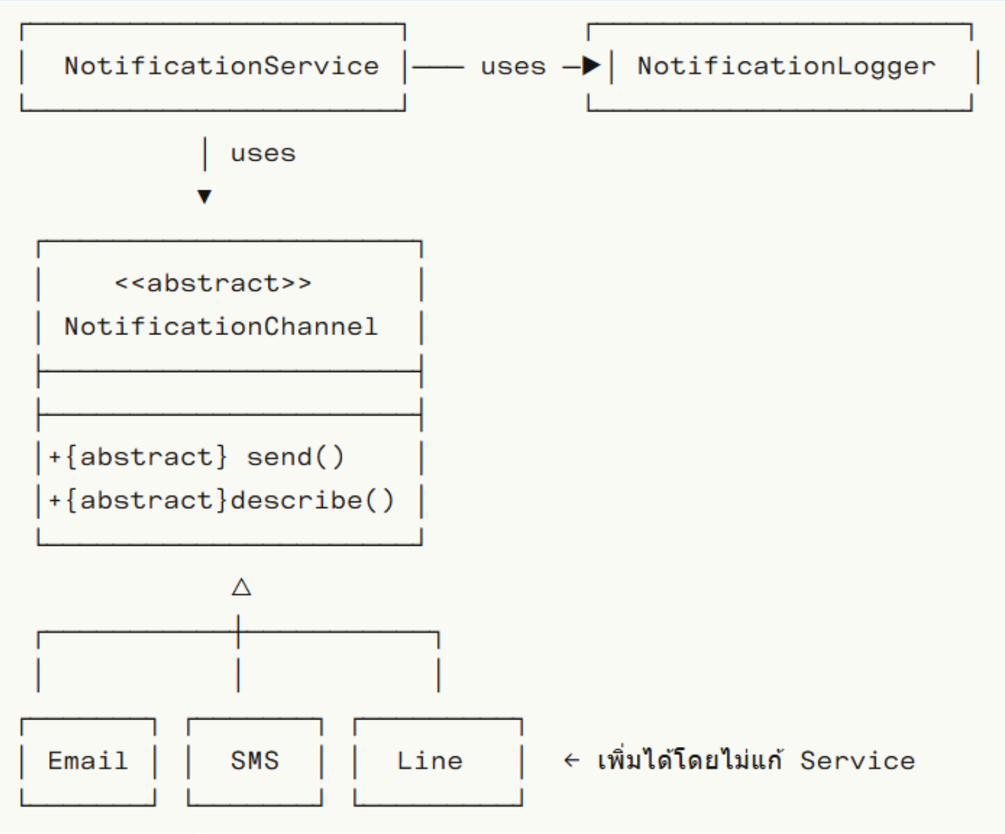
ขั้นที่ 2 — เพิ่มชั้น Abstraction สำหรับช่องทางการส่ง (แก้ OCP)



ขั้นตอนนี้เปลี่ยน NotificationService จากที่เคยตัดสินใจเองว่าจะ format ข้อความอย่างไรตาม string channel ให้กลายเป็นการรับ NotificationChannel (สัญญากลาง) เข้ามาแทน

UML ของ Design — สร้างทีละขั้น

ขั้นที่ 3 — เติม concrete channel ทั้งหมดเข้าไปใต้ Abstract Class



- เส้น uses สองเส้นจาก NotificationService แสดงว่า service รู้จักแค่ “สัญญา” ของทั้ง logger และ channel เท่านั้น
  - ไม่เคยรู้จัก concrete class ตัวใดตัวหนึ่งโดยตรงเลย
- ส่วนพื้นที่ว่างใต้แถว Email, SMS, Line (เหมือนที่เห็นในหัวข้อ OCP) สื่อว่าเพิ่ม channel ใหม่ได้อีกไม่จำกัดโดยไม่กระทบโครงสร้างด้านบน

W1

Array & Dynamic Array

Data Structures & Object-Oriented Programming



Table of Contents

- Linear Data Structures
- Memory Layout และ O(1) Access
- Static vs Dynamic Array และ Doubling Strategy
- Complexity Summary

Linear Data Structures

Linear Data Structures

- นิสิตได้เรียนรู้วิธี “ออกแบบ class” รู้จักสร้าง object ที่มีข้อมูลและพฤติกรรมของตัวเอง (Class, Encapsulation) รู้จักต่อยอด class ให้ทำงานร่วมกันได้ (Inheritance, Polymorphism, Aggregation, Composition) และรู้จักหลักการออกแบบที่ดี (SOLID)
- Phase 2: Linear Data Structures
  - จากเดิมที่ถามว่า “จะออกแบบ class ให้ดีได้อย่างไร”
  - กลายเป็นคำถามใหม่ว่า “จะสร้างโครงสร้างข้อมูล (data structure) ขึ้นมาเองตั้งแต่ต้นได้อย่างไร”
- ทุกเครื่องมือที่เรียนมาตลอด Phase 1 จะถูกนำมาใช้งานจริงในสัปดาห์นี้ เพราะการสร้าง data structure ขึ้นเองก็คือการออกแบบ class ที่มีเป้าหมายเฉพาะเจาะจงกว่าเดิม คือต้องจัดการกับการเก็บและเข้าถึงข้อมูลจำนวนมากอย่างมีประสิทธิภาพ
- data structure แรกที่จะเรียนคือ Array ซึ่งเป็นโครงสร้างข้อมูลพื้นฐานที่สุดและสำคัญที่สุดในวิทยาการคอมพิวเตอร์ เพราะแทบทุกโครงสร้างข้อมูลที่ซับซ้อนกว่านี้ล้วนสร้างขึ้นบนพื้นฐานความเข้าใจเรื่อง array ทั้งสิ้น

“ความเร็ว” ของโปรแกรมวัดกันอย่างไร

- เวลาเราพูดว่าโปรแกรมหนึ่ง “เร็ว” หรือ “ช้า” เราไม่ได้วัดเป็นวินาทีจริงๆ เพราะคอมพิวเตอร์แต่ละเครื่องเร็วไม่เท่ากัน
- สิ่งที่เราสนใจจริงๆ คือ “เมื่อข้อมูลมีขนาดใหญ่ขึ้น เวลาที่ใช้จะเพิ่มขึ้นตามสัดส่วนอย่างไร”
  - สัญกรณ์ที่ใช้บอกความสัมพันธ์นี้เรียกว่า Big-O notation
- O(1) อ่านว่า “โอ ของ 1” หมายถึง เวลาคงที่ ไม่ว่าข้อมูลจะมีกี่ตัวก็ตาม การทำงานนั้นใช้เวลาเท่าเดิมเสมอ
- O(n) อ่านว่า “โอ ของ เอ็น” (n แทนจำนวนข้อมูล) หมายถึง เวลาที่เพิ่มขึ้นตามสัดส่วนโดยตรงกับจำนวนข้อมูล
  - ถ้าข้อมูลมากขึ้นเป็นสองเท่า เวลาก็เพิ่มเป็นสองเท่า
- $O(n^2)$ อ่านว่า “โอ ของ เอ็นกำลังสอง” หมายถึงเวลาที่เพิ่มขึ้นแบบยกกำลังสองตามจำนวนข้อมูล
  - ถ้าข้อมูลมากขึ้นเป็นสองเท่า เวลาเพิ่มขึ้นถึงสี่เท่า ซึ่งเป็นความเร็วที่แย่มากเมื่อข้อมูลมีจำนวนมาก
- หลักการสำคัญที่ต้องจำไว้เสมอคือ
  - O(1) ดีกว่า O(n) เสมอ เมื่อข้อมูลมีจำนวนมาก เพราะ O(1) ไม่สนใจจำนวนข้อมูลเลย
  - O(n) ช้าลงเรื่อยๆ ตามข้อมูลที่เพิ่มขึ้น

ทำไมต้องสร้าง Data Structures เอง?

- ที่ผ่านมา นิสิตใช้ Python list อยู่ตลอดเวลา แต่ยังอาจจะไม่เข้าใจว่า “ทำไม” มันถึงทำงานได้แบบนั้น
  - เมื่อไรควรใช้ list และเมื่อไรควรใช้โครงสร้างอื่นแทน

- **คำถามที่ 1 — ทำไม my_list[0] กับ my_list[999] ใช้เวลาเท่ากัน?**
  - มี list ที่มีสมาชิก 1,000 ตัว ถ้าต้องการอ่านค่าที่ index 0 (ตัวแรก) กับอ่านค่าที่ index 999 (ตัวสุดท้าย)
  - บางคนอาจคิดว่า “การไปถึงตัวสุดท้ายต้องใช้เวลานานกว่า เพราะต้องนับผ่านตัวก่อนหน้าไปเรื่อยๆ ก่อน”
- แต่ในความเป็นจริง my_list[0] และ my_list[999] ใช้เวลาเท่ากัน ไม่มีการนับผ่านตัวก่อนหน้าเลยแม้แต่นิดเดียว
  - คำตอบของคำถามนี้คือหัวใจของหัวข้อ Memory Layout และ O(1) Access ที่จะอธิบายในส่วนถัดไป

ทำไมต้องสร้าง Data Structures เอง? (2)

คำถามที่ 2 — ทำไม Python list ถึงขยายตัวได้โดยอัตโนมัติ?

- เวลาเขียนโค้ด my_list = [] แล้วเรียก .append() ไปเรื่อยๆ ไม่มีใครเคยบอก Python ล่วงหน้าว่า list นี้จะมีสมาชิกกี่ตัว แต่ list ก็ขยายขนาดรับสมาชิกใหม่ได้เสมอ เหมือนมีพื้นที่ไม่จำกัด

- คำถามคือ กลไกเบื้องหลังนี้ทำงานอย่างไร และ“ขยายได้ฟรี” จริง หรือมีต้นทุนซ่อนอยู่ที่มองไม่เห็น
  - คำตอบของคำถามนี้คือ หัวข้อ Doubling Strategy และ Amortized Analysis ที่จะอธิบายในส่วนถัดไป

ทำไมต้องสร้าง Data Structures เอง? (2)

คำถามที่ 3 — เมื่อไรควรใช้ list และเมื่อไรควรใช้โครงสร้างอื่น?

- list ไม่ใช่คำตอบสำหรับทุกสถานการณ์ และการเลือกโครงสร้างข้อมูลผิดสามารถทำให้โปรแกรมช้าลงได้หลายร้อยหรือหลายพันเท่า โดยที่ syntax ยังคงถูกต้องทุกประการและไม่มี error ใดๆ เตือน
- การเรียน Data Structures ไม่ได้มีเป้าหมายเพื่อสร้าง list ขึ้นมาใหม่แข่งกับของจริง เพราะ Python list ที่มีอยู่ถูก optimize มาอย่างดีเยี่ยมโดยทีมพัฒนา Python เอง
- เป้าหมายที่แท้จริงคือ
  - เข้าใจเครื่องมือที่ใช้อยู่ทุกวันอย่างลึกซึ้ง แทนที่จะใช้มันแบบไม่รู้กลไกเบื้องหลัง
  - ฝึกทักษะ “เลือก” โครงสร้างข้อมูลให้ถูกต้องกับสถานการณ์ เพราะในการทำงานจริง จะมีสถานการณ์ที่ built-in ของภาษาใช้ไม่ได้ หรือมีโครงสร้างข้อมูลเฉพาะทางที่เหมาะกับปัญหานั้นมากกว่า
  - ฝึกความสามารถในการ implement โครงสร้างข้อมูลตั้งแต่ต้น ซึ่งเป็นทักษะที่แสดงถึงความเข้าใจ programming อย่างลึกซึ้ง และเป็นพื้นฐานสำคัญสำหรับการเรียนวิชา Algorithm และ Data Structures ขั้นสูงในอนาคต

เชื่อมโยงกับ OOP ที่เรียนมา

- DynamicArray ที่จะสร้างในสัปดาห์นี้ คือ class ธรรมดาที่ใช้หลักการ Encapsulation อย่างเข้มข้น
- ข้อมูลภายใน (_capacity, _size, _data) จะถูกจัดการอย่างระมัดระวังผ่าน method ต่างๆ
- มี method พิเศษที่ขึ้นต้นและลงท้ายด้วยขีดล่างสองอัน (เรียกว่า dunder methods หรือ magic methods)
  - __len__, __getitem__, __setitem__ จะเข้ามามีบทบาทสำคัญ
  - เพราะ method เหล่านี้คือกลไกที่ทำให้ object ที่สร้างขึ้นเองสามารถใช้ syntax คุ้นเคยอย่าง len(arr), arr[i], arr[i] = value ได้
- ในหัวข้อถัดไป จะเริ่มตอบคำถามแรก คือ ทำไม array ถึงเข้าถึงข้อมูลได้เร็วคงที่ (O(1)) โดยจะลงลึกไปถึงระดับว่าคอมพิวเตอร์เก็บข้อมูลในหน่วยความจำอย่างไรจริงๆ

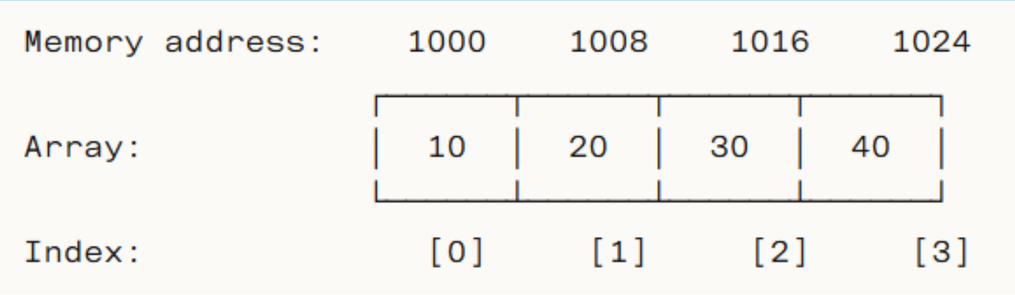
Memory Layout has O(1) Access

หน่วยความจำ (Memory) คืออะไร

- ก่อนจะเข้าใจว่า array ทำงานอย่างไร ต้องเข้าใจก่อนว่าคอมพิวเตอร์เก็บข้อมูลไว้ที่ไหนและอย่างไร
- จินตนาการว่าหน่วยความจำ (RAM) ของคอมพิวเตอร์ เป็นเหมือน ตู้ล็อกเกอร์ยาวเหยียดที่มีช่องนับล้านช่องเรียงต่อกัน แต่ละช่องมีหมายเลขกำกับไว้อย่างชัดเจน เรียกว่า memory address เหมือนหมายเลขล็อกเกอร์ที่ไม่ซ้ำกันเลย
- เมื่อโปรแกรมต้องการเก็บข้อมูลสักตัว คอมพิวเตอร์จะ “จอง” ช่องล็อกเกอร์บางช่องให้ และข้อมูลนั้นก็จะถูกเก็บอยู่ที่ตำแหน่งนั้นในหน่วยความจำ ตราบใดที่โปรแกรมยังต้องใช้งานอยู่

Contiguous Memory

- Array คือการจัดเก็บข้อมูลใน หน่วยความจำที่ต่อเนื่องกัน (contiguous memory)
  - แต่ละ element เก็บอยู่ในช่องล็อกเกอร์ที่ติดกันสนิท ไม่มีช่องว่างคั่นระหว่างกันเลย
- ตัวอย่าง array ของจำนวนเต็ม 4 ตัว



- address ของแต่ละ element ห่างกันเท่าๆ กันเสมอ
  - ในที่นี้ห่างกันทีละ 8 (จาก 1000 ไป 1008 ไป 1016 ไป 1024)
- ลำดับของ element ในหน่วยความจำตรงกับลำดับ index
  - element ที่ index 0 อยู่ที่ address น้อยที่สุด และไล่เพิ่มขึ้นตาม index
- ในระบบคอมพิวเตอร์แบบ 64-bit พื้นที่สำหรับเก็บตัวเลขอ้างอิงหนึ่งตัว (เรียกว่า pointer) ใช้พื้นที่ 8 bytes เสมอ
  - ดังนั้น element ถัดไปจึงอยู่ห่างจาก element ปัจจุบันไป 8 ตำแหน่งอย่างสม่ำเสมอ

สูตรคณิตศาสตร์ที่ทำให้ Array เร็ว

- เพราะ elements เรียงติดกันเป็นระเบียบ คอมพิวเตอร์จึงไม่จำเป็นต้องค้นหา ตำแหน่งของ element ใดๆ เลย
- มันสามารถ คำนวณ ตำแหน่งได้ทันทีด้วยสูตรเดียว

```
address_of_element[i] = base_address + (i × element_size)
```

- โดย base_address คือตำแหน่งของ element ตัวแรก (index 0)
- element_size คือ ขนาดของแต่ละ element (ในที่นี้คือ 8 bytes)

- จากตัวอย่างข้างต้น สมมติ base_address = 1000 และ element_size = 8
  - element[0] อยู่ที่ 1000 + (0 × 8) = 1000
  - element[2] อยู่ที่ 1000 + (2 × 8) = 1016
  - element[999] อยู่ที่ 1000 + (999 × 8) = 8992

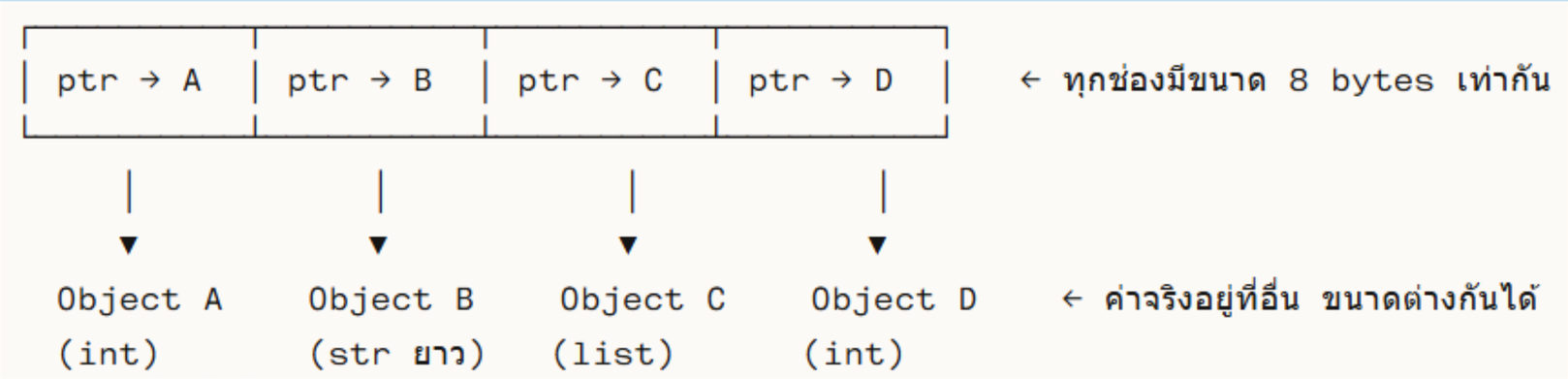
ทำไม my_list[0] กับ my_list[999] ใช้เวลาเท่ากัน

- ไม่ว่า index จะเป็น 0 หรือ 999 หรือ 999999 คอมพิวเตอร์ทำ ขั้นตอนเดียวกันทุกประการ คือ
  - เอาค่า base_address ที่รู้อยู่แล้ว
  - เอา index คูณด้วย element_size
  - บวกสองค่านั้นเข้าด้วยกัน
  - ไปอ่านข้อมูลที่ address นั้นตรงๆ
- การคูณและการบวกเป็นการคำนวณที่ใช้เวลาคงที่เสมอ ไม่ว่าตัวเลขจะเล็กหรือใหญ่แค่ไหน ไม่มีการวนลูป
- นี่คือเหตุผลที่การเข้าถึง array ด้วย index จึงเป็น O(1) — เวลาคงที่ ไม่ขึ้นกับขนาดของ array



การทำงานของ Python list

- ใน array แบบดั้งเดิม ทุก element ต้องมีขนาดเท่ากัน เพราะสูตร base_address + (i × element_size) ต้องใช้ element_size เป็นค่าคงที่ตัวเดียว
- แต่ปัญหาคือใน Python ตัวแปรหนึ่งตัวเก็บอะไรก็ได้ — เลขจำนวนเต็ม, string, หรือ object
  - ขนาดของแต่ละอย่างไม่เท่ากันเลย
- Python จะใช้สูตรคำนวณ address แบบตรงไปตรงมาได้อย่างไร
  - Python ไม่ได้เก็บค่าจริงไว้ในช่อง array โดยตรง แต่เก็บ pointer (ตัวชี้) ซึ่งเป็นตัวเลขอ้างอิงขนาดคงที่ 8 bytes เท่ากันทุกตัว ที่ ชี้ไปยังตำแหน่งอื่นในหน่วยความจำที่เก็บค่าจริงอยู่ อีกทีหนึ่ง

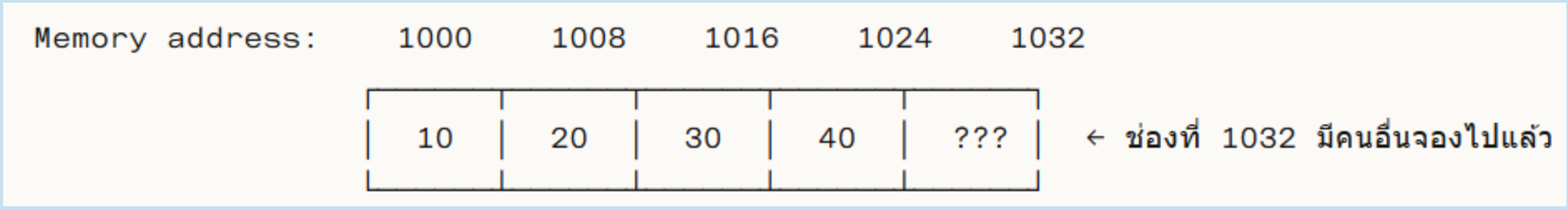


- หลังจากคำนวณ address เจอแล้ว ต้องมีขั้นตอนเพิ่ม คือตาม pointer นั้นไปอ่านค่าจริงอีกที
- แต่ขั้นตอนพิเศษนี้ก็ยังคงเป็นเวลาคงที่เช่นกัน
- ดังนั้นโดยรวมแล้ว การเข้าถึง element ของ Python list ยังคงเป็น O(1)

Static vs Dynamic Array และ Doubling Strategy

ปัญหาของ Array แบบดั้งเดิม: ขนาดต้องคงที่

- array เร็วเพราะข้อมูลเรียงติดกันในหน่วยความจำ แต่ความ “เรียงติดกัน” นี้เองที่สร้างปัญหาใหญ่
- ลองจินตนาการว่ามี array ขนาด 4 ช่องอยู่ในหน่วยความจำ เรียงติดกัน



- ถ้าต้องการเพิ่ม element ตัวที่ 5 เข้าไป จะเกิดปัญหาทันที เพราะช่องถัดจาก array นี้ (ตำแหน่ง 1032) อาจถูกโปรแกรมอื่นหรือข้อมูลอื่นจองไปแล้ว
  - ไม่มีทางรู้ล่วงหน้าว่าช่องถัดไปจะว่างอยู่หรือไม่ เพราะ OS เป็นผู้จัดสรรหน่วยความจำให้โปรแกรมต่างๆ อย่างอิสระ
- ข้อจำกัดพื้นฐานที่สุดของ array แบบดั้งเดิม : เมื่อจองพื้นที่ไปแล้ว ขนาดของมันจะขยับขยายออกไปทางขวาไม่ได้อีก

Static Array — ขนาดคงที่ตั้งแต่ต้น

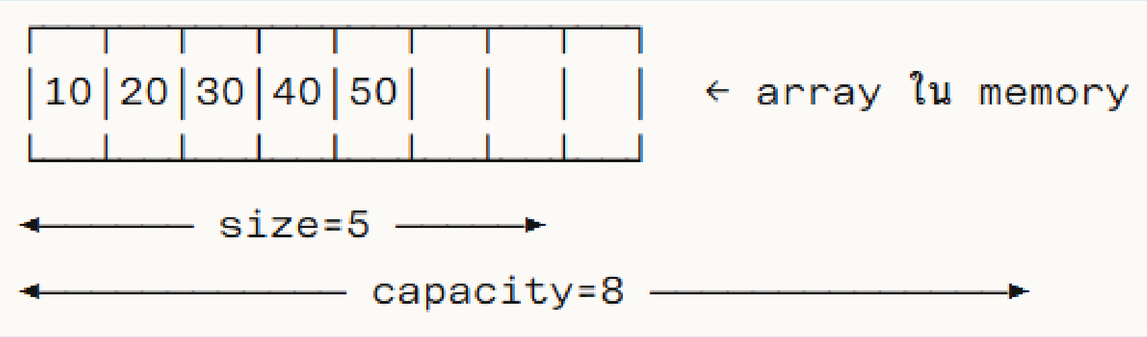
- Static Array คือ array ที่มีขนาดคงที่ กำหนดไว้ตั้งแต่ตอนสร้าง และไม่สามารถเปลี่ยนแปลงได้อีกเลยตลอดอายุการใช้งาน
  - ถ้าต้องการเพิ่ม element เกินขนาดที่กำหนดไว้ โปรแกรมจะเกิด error ทันที
- ภาษา C ใช้ static array เป็นค่าเริ่มต้น ผู้เขียนโปรแกรมต้องกำหนดขนาดล่วงหน้าอย่างชัดเจน เช่น “ต้องการ array ที่เก็บตัวเลขได้ 100 ตัว” และถ้าวันหนึ่งต้องการเก็บตัวที่ 101 ก็ต้องหาวิธีจัดการเอง เช่น สร้าง array ใหม่ที่ใหญ่กว่าแล้วย้ายข้อมูลทั้งหมดไป
- ข้อดีของ static array คือ เรียบง่ายและคาดเดาได้
- ข้อเสีย คือ ผู้เขียนโปรแกรมต้องรู้ล่วงหน้าว่าจะมีข้อมูลกี่ตัว ซึ่งในความเป็นจริงมักไม่รู้ล่วงหน้า

Dynamic Array — ขยายขนาดได้เมื่อจำเป็น

- Dynamic Array คือ array ที่สามารถขยายขนาดได้โดยอัตโนมัติเมื่อจำเป็น โดยผู้ใช้งานไม่ต้องกังวลเรื่องขนาดล่วงหน้า
  - Python list, Java ArrayList, และ C++ vector ล้วนเป็น dynamic array ทั้งสิ้น
- คำว่า “ขยายได้” ไม่ได้แปลว่า “ไม่มีต้นทุน”
  - เบื้องหลังกลไกที่ทำให้ list ขยายขนาดได้ มีขั้นตอนที่ซับซ้อนและมีค่าใช้จ่ายซ่อนอยู่

Size ไม่เท่ากับ Capacity

- **size คือ จำนวน element ที่เก็บอยู่จริงในขณะนี้** — นับเฉพาะของจริง
- capacity คือ จำนวนช่อง (slot) ทั้งหมดที่จองไว้ในหน่วยความจำแล้ว รวมถึงส่วนที่ยังว่างอยู่ด้วย
- ความสัมพันธ์ระหว่างสองค่านี้เขียนเป็นกฎได้ว่า size ≤ capacity เสมอ



- จากภาพนี้ มี element จริงอยู่ 5 ตัว (size=5) แต่จองพื้นที่ไว้ล่วงหน้าถึง 8 ช่อง (capacity=8) ส่วนที่เหลืออีก 3 ช่องยังว่างอยู่ รอไว้สำหรับ element ที่จะเพิ่มเข้ามาในอนาคตโดยไม่ต้องขอพื้นที่ใหม่
- เหตุผลที่ต้อง “จองเผื่อ” แบบนี้ คือถ้าจองพอดีกับจำนวนที่ใช้จริงทุกครั้ง การเพิ่ม element ใหม่แม้แต่ตัวเดียวก็ต้องขอพื้นที่ใหม่ทุกครั้งไป ซึ่งมีค่าใช้จ่ายสูงมาก
  - การจองเผื่อไว้ล่วงหน้าจึงเป็นกลยุทธ์ที่ช่วยลดจำนวนครั้งที่ต้องขอพื้นที่ใหม่

Doubling Strategy — กลไกการขยายตัวที่แท้จริง

เมื่อไรก็ตามที่ size == capacity (ช่องที่จองไว้เต็มหมดแล้ว) และมีคนพยายาม append element ใหม่เข้าไปอีก dynamic array จะทำตามขั้นตอนต่อไปนี้

- **ขั้นที่ 1 — จอง (Allocate) memory ใหม่**ก้อนหนึ่ง ขนาดเท่ากับ capacity × 2 (สองเท่าของเดิม)
- **ขั้นที่ 2 — คัดลอก (Copy) elements ทั้งหมด**จาก array เก่า ไปยัง array ใหม่ที่เพิ่งจองไว้
- **ขั้นที่ 3 — ปล่อย (Release) memory เก่า**ทิ้งไป เพราะไม่ใช้แล้ว
- **ขั้นที่ 4 — เพิ่ม (Add) element ใหม่ที่ต้องการ** append เข้าไปใน array ใหม่นี้
- **ขั้นที่ 5 — อัปเดต ค่า capacity ให้เป็นค่าใหม่** (สองเท่าของเดิม)

กระบวนการนี้เรียกว่า resize (การปรับขนาด) และเกิดขึ้นแบบ “ไม่มีใครเห็น”

จากมุมมองของคนที่เขียนโค้ดเรียก .append() เขาจะเห็นแค่ว่า element ถูกเพิ่มเข้าไปเรียบร้อย โดยไม่รู้เลยว่าเบื้องหลังมีการจองพื้นที่ใหม่และคัดลอกข้อมูลทั้งหมดเกิดขึ้น

จำลองการ Append ตั้งแต่เริ่มต้น

เมื่อ append ค่า 10, 20, 30, 40, 50 เข้าไปทีละตัว โดยเริ่มจาก capacity = 1

```
capacity=1  [10]                          size=1, capacity=1

append(20) → เต็มพอดี (size==capacity) → RESIZE (×2)
            [10|__]  ← array ใหม่         size=1, capacity=2 (ระหว่าง copy)
            [10|20]                       size=2, capacity=2

append(30) → เต็มพอดี → RESIZE (×2)
            [10|20|__|__]                 size=2, capacity=4 (ระหว่าง copy)
            [10|20|30|__]                 size=3, capacity=4

append(40) → ยังไม่เต็ม (size < capacity) → ไม่ต้อง resize
            [10|20|30|40]                 size=4, capacity=4

append(50) → เต็มพอดี → RESIZE (×2)
            [10|20|30|40|__|__|__|__]     size=4, capacity=8 (ระหว่าง copy)
            [10|20|30|40|50|__|__|__]     size=5, capacity=8
```

capacity ไล่จาก 1 → 2 → 4 → 8 เพิ่มเป็นสองเท่าทุกครั้งที่ต้อง resize

ไม่ใช่ทุกครั้งที่ append จะต้อง resize อย่างตอน append(40) ที่ size ยังน้อยกว่า capacity อยู่ (3 < 4) จึงเพิ่มเข้าไปตรงๆ ได้เลยโดยไม่ต้องจองพื้นที่ใหม่

ทำไมต้อง “เพิ่มเป็นสองเท่า” ไม่ใช่ “บวกทีละค่าคงที่”

สมมติต้องการ append ทั้งหมด n ครั้ง

กลยุทธ์ที่ 1 — บวกทีละ 1 ทุกครั้งที่เต็ม (capacity ไล่ 1 → 2 → 3 → 4 → 5 → … → n)

- ทุกครั้งที่ resize ต้อง copy element ทั้งหมดที่มีอยู่ในตอนนั้น ลองนับดูว่า resize แต่ละครั้ง copy กี่ตัว
- resize ครั้งที่ 1 (จาก capacity 1 เป็น 2) ต้อง copy 1 ตัว
- resize ครั้งที่ 2 (จาก capacity 2 เป็น 3) ต้อง copy 2 ตัว
- resize ครั้งที่ 3 (จาก capacity 3 เป็น 4) ต้อง copy 3 ตัว
- …ไปเรื่อยๆ จนถึง resize ครั้งที่ n-1 ต้อง copy (n-1) ตัว
- ต้นทุนรวมของการ copy ทั้งหมด = 1 + 2 + 3 + … + (n-1) ซึ่งมีค่าประมาณ $n^2/2$ ต้นทุนรวมเป็น $O(n^2)$ — ยิ่ง n มากเท่าไร ต้นทุนยิ่งพุ่งขึ้นแบบยกกำลังสอง ซึ่งแย่มาก

กลยุทธ์ที่ 2 — เพิ่มเป็นสองเท่าทุกครั้งที่เต็ม (capacity ไล่ 1 → 2 → 4 → 8 → 16 → …)

- resize เกิดขึ้นเพียง $\log_2(n)$ ครั้งเท่านั้น (เพราะ capacity โตแบบทวีคูณ ไม่ใช่ทีละหนึ่ง) และต้นทุนรวมของการ copy ทั้งหมดคือ $1 + 2 + 4 + 8 + \ldots \leq 2n$
- ต้นทุนรวมของการ copy ทั้งหมดเป็น **O(n)** เท่านั้น — เร็วกว่ากลยุทธ์แรกอย่างมหาศาลเมื่อ n มีค่ามาก

เปรียบเทียบด้วยตัวเลขที่จับต้องได้

ลองแทนค่า n = 1,000

- กลยุทธ์บวกทีละ 1 ต้องใช้ต้นทุนการ copy รวมประมาณ $1000^2 / 2 = 500{,}000$ ครั้ง
- กลยุทธ์เพิ่มเป็นสองเท่า ต้องใช้ต้นทุนการ copy รวมไม่เกิน $2 \times 1000 = 2{,}000$ ครั้ง

- ความต่างคือ 250 เท่า

- จากตัวเลขนี้จะเห็นได้ชัดว่าทำไม doubling strategy ถึงเป็นทางเลือกที่ถูกใช้ในภาษาโปรแกรมมิ่งเกือบทุกภาษา (แม้ตัวเลขที่แน่นอนอาจต่างกันไปบ้าง แต่หลักการพื้นฐานคือ “เพิ่มแบบทวีคูณ” ไม่ใช่ “เพิ่มแบบบวกทีละค่าคงที่”)

Complexity Summary

Insert และ Delete ที่ต้นหรือกลาง Array

- elements ของ array ต้องเรียงติดกันในหน่วยความจำเสมอ ทำให้ access เป็น O(1) ได้ แต่กลายเป็นปัญหาใหญ่เมื่อต้องแทรกหรือลบ element ที่ไม่ใช่ตำแหน่งสุดท้าย
- ลองพิจารณาสถานการณ์ที่ต้องการ แทรก ค่า 99 เข้าไปที่ตำแหน่ง index 1 ของ array [10, 20, 30, 40]

```
ก่อนแทรก:   [10|20|30|40|   |   ]      size=4

ต้องการ:     [10|99|20|30|40|   ]      ← แทรก 99 ที่ index 1
```

จำเป็นต้อง “เลื่อน” elements ทั้งหมดที่อยู่ตั้งแต่ตำแหน่งที่จะแทรกเป็นต้นไปให้ขยับไปทางขวาทีละช่อง เพื่อเปิดที่ว่างให้ค่าใหม่เข้าไปแทรกได้

Insert และ Delete ที่ต้นหรือกลาง Array

```
ก่อนแทรก:   [10|20|30|40|   |   ]     size=4

ต้องการ:    [10|99|20|30|40|   ]     ← แทรก 99 ที่ index 1
```

```
ขั้นที่ 1: เลื่อน 40 จาก index 3 ไป index 4
           [10|20|30|   |40]

ขั้นที่ 2: เลื่อน 30 จาก index 2 ไป index 3
           [10|20|   |30|40]

ขั้นที่ 3: เลื่อน 20 จาก index 1 ไป index 2
           [10|   |20|30|40]

ขั้นที่ 4: ใส่ 99 ลงใน index 1 ที่ว่างแล้ว
           [10|99|20|30|40]
```

- ยิ่งแทรกใกล้ ต้น array มากเท่าไร ยิ่งต้องเลื่อน elements มากขึ้นเท่านั้น

- ในกรณีที่แย่ที่สุด (แทรกที่ตำแหน่งแรกสุด index 0) ต้องเลื่อน elements ทั้งหมด ที่มีอยู่ ซึ่งมีจำนวน n ตัว ดังนั้น Insert ที่ต้นหรือกลาง array จึงเป็น O(n)
- หลักการเดียวกันนี้ใช้ได้กับ Delete ที่ต้นหรือกลาง array เช่นกัน แต่กลับทิศทาง
  - เมื่อลบ element ออกจากตำแหน่งใดตำแหน่งหนึ่ง ต้อง เลื่อน elements ที่อยู่ทางขวาของตำแหน่งนั้นทั้งหมดมาทางซ้าย เพื่อปิดช่องว่างที่เกิดขึ้น ซึ่งก็เป็น O(n) เช่นเดียวกัน

Delete ท้าย Array เป็น O(1)

- การลบ element ตัวสุดท้าย ของ array ไม่จำเป็นต้องเลื่อนอะไรเลยแม้แต่ตัวเดียว เพราะไม่มี element ใดอยู่ทางขวาของตำแหน่งสุดท้าย

```
ก่อนลบ:    [10|20|30|40|   ]   size=4

หลังลบ:    [10|20|30|   |   ]   size=3   ← แค่ลด size ลง ไม่ต้องเลื่อนอะไรเลย
```

- การลบท้ายเพียงแค่ ลดค่า size ลงหนึ่งค่า
- การกระทำนี้ใช้เวลาคงที่ ไม่ว่า array จะมีขนาดใหญ่แค่ไหน จึงเป็น O(1)

Code Implementation

Dunder Methods คืออะไร

- Python มีระบบที่เรียกว่า dunder methods (ย่อมาจาก “double underscore”)
  - method ที่ขึ้นต้นและลงท้ายด้วยขีดล่างสองอัน เช่น __init__, __len__
  - หรือบางครั้งเรียกว่า magic methods
- ระบบนี้ทำหน้าที่เป็น “สะพานเชื่อม” ระหว่าง syntax ที่คุ้นเคย กับ method จริงที่ทำงานเบื้องหลัง

Syntax ที่เขียน	Python แปลงเป็นการเรียก	ความหมาย
`len(arr)`	`arr.__len__()`	ถามว่า arr มีกี่ element
`arr[i]`	`arr.__getitem__(i)`	อ่านค่าที่ index i
`arr[i] = v`	`arr.__setitem__(i, v)`	เขียนค่า v ลงที่ index i
`str(arr)` หรือ `print(arr)`	`arr.__str__()`	แปลงเป็น string อ่านง่ายสำหรับผู้ใช้
`repr(arr)`	`arr.__repr__()`	แปลงเป็น string สำหรับ debug

โครงสร้างโดยรวม

```
class DynamicArray:
    """
    Dynamic Array ที่ implement doubling strategy
    ใช้ Python list ขนาดคงที่เป็น underlying storage

    Invariants:
        0 <= _size <= _capacity
        _data มีความยาวเท่ากับ _capacity เสมอ
    """
```

- คำว่า invariant หมายถึง “เงื่อนไขที่ต้องเป็นจริงเสมอ ตลอดเวลาที่ object นี้มีชีวิตอยู่”
- ไม่ว่าจะเรียก method ไหนของ DynamicArray ก็ตาม เมื่อ method นั้นทำงานเสร็จแล้ว เงื่อนไขสองข้อนี้ต้องยังคงเป็นจริงเสมอ นี่คือ “สัญญา” ที่ผู้เขียน class ต้องรักษาไว้ให้ได้ตลอดทุก method
- การตั้งชื่อ attribute ทั้ง 3 ตัว (_capacity, _size, _data) ขึ้นต้นด้วย underscore เดี่ยว (_) คือเป็น protected attribute
    - บอกผู้ใช้งานว่า “ไม่ควรเข้าถึงโดยตรงจากภายนอก ควรผ่าน method ที่เปิดให้เท่านั้น” (เช่น len(), indexing, หรือ append())

init — จุดเริ่มต้นของทุก object

```
def __init__(self):
    self._capacity = 1          # จำนวน slot ที่จองไว้ใน memory
    self._size     = 0          # จำนวน element ที่มีจริง
    self._data     = [None] * self._capacity
```

- self._capacity = 1 — เริ่มต้นจองพื้นที่ไว้แค่ 1 ช่องเท่านั้น (ไม่จองเผื่อเยอะ เพราะยังไม่รู้ว่าจะมีข้อมูลกี่ตัว)
- self._size = 0 — ยังไม่มี element ใดๆ อยู่จริงเลย
- self._data = [None] * self._capacity
  - สร้าง Python list ที่มีความยาวเท่ากับ _capacity แล้วเติมด้วยค่า None ไว้ก่อนทั้งหมด เพื่อจอง “ที่ว่าง” ไว้ล่วงหน้า
  - _data คือ Python list ธรรมดาที่ใช้เป็นพื้นที่เก็บข้อมูลจริง (underlying storage)
- DynamicArray เองเป็นผู้ควบคุม logic ของ _size/_capacity/_resize ทั้งหมดด้วยตัวเอง

len — ตอบคำถาม “มีกี่ตัว”

```
def __len__(self) -> int:
    return self._size
```

- ต้องคืนค่า _size ไม่ใช่ len(self._data) เพราะ _data มีความยาวเท่ากับ _capacity เสมอ (รวมช่องว่างที่ยังไม่ได้ใช้)
- ถ้าคืนค่าความยาวของ _data ตรงๆ จะได้ค่า capacity ซึ่งผิดความหมายที่ต้องการ

getitem และ setitem

```
def __getitem__(self, index: int):
    if not (0 <= index < self._size):
        raise IndexError(
            f"Index {index} out of range (size={self._size})"
        )
    return self._data[index]

def __setitem__(self, index: int, value) -> None:
    if not (0 <= index < self._size):
        raise IndexError(
            f"Index {index} out of range (size={self._size})"
        )
    self._data[index] = value
```

- __getitem__ อ่านค่า
- __setitem__ เขียน ค่าใหม่
- if not (0 <= index < self._size): คือการตรวจสอบขอบเขต เรียกว่า bounds checking
  - “ถ้า index ไม่ได้อยู่ในช่วงที่ถูกต้อง (ไม่ใช่ตั้งแต่ 0 จนถึงน้อยกว่า size) ให้ raise error ทันที”
- การตรวจสอบเทียบกับ _size ในทาง logic ของ DynamicArray แล้ว ช่องที่เกิน _size ไปถือว่า “ไม่มีข้อมูลอยู่จริง” ผู้ใช้งานจึงไม่ควรเข้าถึงช่องเหล่านั้นได้เลย
- การป้องกันด้วย bounds checking ทำให้ DynamicArray มี behavior ที่ถูกต้องตรงกับ Python list ที่ raise IndexError เมื่อเข้าถึง index เกินขอบเขต

str กับ repr

```
def __str__(self) -> str:
    elements = [str(self._data[i]) for i in range(self._size)]
    return f"[{', '.join(elements)}]"

def __repr__(self) -> str:
    return f"DynamicArray(size={self._size}, capacity={self._capacity})"
```

- ทั้งสอง method นี้ทำหน้าที่ “แปลง object เป็น string”
- __str__ ถูกเรียกอัตโนมัติเมื่อใช้ print(arr) หรือ str(arr)
  - โค้ดนี้ตั้งใจให้แสดงผลคล้าย list ทั่วไป เช่น [10, 20, 30, 40, 50] โดยใช้ เฉพาะ _size elements แรกเท่านั้นไม่แสดงค่า None ที่เป็นช่องว่างใน capacity เลย
- __repr__ ถูกออกแบบมาเพื่อช่วย debug ถูกเรียกเมื่อใช้ repr(arr) หรือเมื่อพิมพ์ object นั้นตรงๆ ใน Python interactive shell
  - รูปแบบที่เขียนไว้ "DynamicArray(size=X, capacity=Y)" แสดงทั้ง size และ capacity พร้อมกัน ซึ่งมีประโยชน์มากตอนกำลังพัฒนาและทดสอบ เพราะทำให้เห็นสถานะภายในทั้งสองค่าพร้อมกันในคราวเดียว ไม่ต้องเรียก len() แยกแล้วเดา capacity เอง

append

```
def append(self, value) -> None:
    """เพิ่ม element ท้าย array --- O(1) amortized"""
    if self._size == self._capacity:
        self._resize()
    self._data[self._size] = value
    self._size += 1
```

- บรรทัดแรก if self._size == self._capacity: คือการเช็คว่า “เต็มพอดีหรือยัง”
  - ถ้า _size เท่ากับ _capacity แปลว่าไม่มีช่องว่างเหลือให้เพิ่มอีกแล้ว จึงต้องเรียก self._resize() ก่อนเพื่อขยายพื้นที่
- บรรทัดที่สอง self._data[self._size] = value คือการใส่ค่าใหม่ลงในช่องที่ว่างอยู่ พอดีที่ตำแหน่ง _size (ตำแหน่งถัดจาก element สุดท้ายที่มีอยู่)
  - มั่นใจได้เสมอว่ามีช่องว่างอยู่จริง เพราะถ้าเต็มก็ผ่านการ resize ไปแล้วในบรรทัดก่อนหน้า
- บรรทัดที่สาม self._size += 1 คือการอัปเดต _size ให้สะท้อนว่าตอนนี้มี element เพิ่มขึ้นมาอีกหนึ่งตัวแล้ว
  - ถ้าลืมเขียนบรรทัดนี้ทุกครั้งที่ append ค่าใหม่จะไปเขียนทับตำแหน่งเดิมซ้ำๆ โดยไม่รู้ตัว

_resize

```
def _resize(self) -> None:
    """Doubling strategy: สร้าง array ใหม่ใหญ่กว่าเดิมสองเท่า --- O(n)"""
    new_capacity = self._capacity * 2
    new_data     = [None] * new_capacity
    for i in range(self._size):       # copy เฉพาะ _size ไม่ใช่ _capacity
        new_data[i] = self._data[i]
    self._data     = new_data
    self._capacity = new_capacity
```

- new_capacity = self._capacity * 2 — คำนวณขนาดใหม่ เป็นสองเท่าของเดิม
- new_data = [None] * new_capacity — จองพื้นที่ใหม่ (สร้าง Python list ใหม่ที่มีความยาวเท่ากับ new_capacity) เติมด้วย None ไว้ก่อน
- for i in range(self._size): — loop วนตาม _size เพราะต้องการคัดลอกเฉพาะข้อมูลของเดิมที่มีอยู่ ไม่ใช่คัดลอกช่องว่าง None ที่ไม่มีความหมายไปด้วย
- self._data = new_data — เปลี่ยนให้ _data ชี้ไปที่ array ใหม่แทนที่ array เก่า (array เก่าที่ไม่มีใครอ้างอิงถึงอีกแล้วจะถูก Python’s garbage collector เก็บกวาดไปเอง แบบอัตโนมัติที่ไม่ต้องเขียนโค้ดเอง)
- self._capacity = new_capacity — อัปเดตค่า _capacity ให้ตรงกับความเป็นจริงใหม่

ทดสอบการทำงาน

```
arr = DynamicArray()
print(repr(arr))                # DynamicArray(size=0, capacity=1)

for val in [10, 20, 30, 40, 50]:
    arr.append(val)
    print(repr(arr))
```

ผลลัพธ์ที่ได้จริง:

```
DynamicArray(size=0, capacity=1)
DynamicArray(size=1, capacity=1)
DynamicArray(size=2, capacity=2)
DynamicArray(size=3, capacity=4)
DynamicArray(size=4, capacity=4)
DynamicArray(size=5, capacity=8)
```

- เริ่มต้น size=0, capacity=1 ตรงตาม __init__
- append 10 เข้าไป size กลายเป็น 1 แต่ capacity ยังคงเป็น 1 (เพราะตอน append นี้ _size (0) ยังไม่เท่ากับ _capacity (1) จึงยังไม่ resize)
- append 20 เข้าไป _size (1) เท่ากับ _capacity (1) พอดี จึงเกิด resize ทำให้ capacity กลายเป็น 2
- ตัวเลข capacity ไล่เป็น 1 → 1 → 2 → 4 → 4 → 8

ทดสอบการทำงาน

```
print(arr)                      # [10, 20, 30, 40, 50]
print(len(arr))                  # 5
print(arr[2])                    # 30

arr[2] = 99
print(arr)                      # [10, 20, 99, 40, 50]

try:
    print(arr[10])
except IndexError as e:
    print(e)                    # Index 10 out of range (size=5)
```

- เมื่อพยายามเข้าถึง arr[10] ทั้งที่มีข้อมูลจริงแค่ 5 ตัว (size=5) ระบบ raise IndexError ทันที แม้ว่า _capacity ตอนนั้นจะเป็น 8
- นี่คือหลักฐานที่ชัดเจนว่า DynamicArray ปกป้องขอบเขตตาม ความหมายเชิงตรรกะ (logical size) ไม่ใช่ตามพื้นที่ทางกายภาพที่จองไว้จริง